1 Contagem de Operações

Instruções: Para cada um dos trechos de código a seguir, determine a função $T(n)$ correspondente à quantidade exata de operações de subtração (−) executadas e apresente a classe de complexidade assintótica em notação Θ.

1.

```
for (int i = 0; i < n; i++) {
    a--;
}
```

2.

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        a--;
    }
}
```

3.

```
for (int i = 1; i <= n; i *= 2) {
    a--;
}
```

4.

```
for (int i = n; i >= 1; i /= 2) {
    a--;
}
```

5.

```
for (int i = n; i >= 1; i /= 3) {
    a--;
}
```

6.

```
//suponha i declarado com int acima
for (i = (i + 1); i <= n - 2; i++) {
    a--;
}
```

7.

```
for (int i = n; i >= 1; i >>= 1) {
    a--;
}
```

8.

```java
for (int i = 1; i < n - 1; i++) {
    for (int j = i; j < n ; j++) {
        a--;
    }
}
```

2 Somatórios

Instruções: Resolva os somatórios (encontrando sua fórmula fechada) listados abaixo.

Folha de Apoio: Propriedades e Identidades Notáveis

Propriedades Gerais:

- $\sum_{i=a}^{b} c = c(b-a+1)$
- $\sum_{i=a}^{b} c \cdot f(i) = c \sum_{i=a}^{b} f(i)$
- $\sum_{i=a}^{b}[f(i) \pm g(i)] = \sum_{i=a}^{b} f(i) \pm \sum_{i=a}^{b} g(i)$

Somas de Potências (1 a m):

- $\sum_{i=1}^{m} i = \frac{m(m+1)}{2}$
- $\sum_{i=1}^{m} i^2 = \frac{m(m+1)(2m+1)}{6}$
- $\sum_{i=1}^{m} i^3 = \left[\frac{m(m+1)}{2}\right]^2$

1. $\sum_{i=1}^{n} 1$

2. $\sum_{i=3}^{n} 4$

3. $\sum_{i=1}^{n-3} 3$

4. $\sum_{i=1}^{n-7} n$

5. $\sum_{i=1}^{n-2}(3n-1)$

6. $\sum_{i=2}^{n-1}(2n)^2$

7. $\sum_{i=3}^{n-2}(3n)^2$

8. $\sum_{i=1}^{n} i$

9. $\sum_{i=1}^{n}(2i+1)$

10. $\sum_{i=2}^{n}(3i-2)$

11. $\sum_{i=1}^{n}(i+1)^2$

12. $\sum_{i=1}^{n}(i-2)^2$

13. $\sum_{i=1}^{n-1} i(i+1)$

14. $\sum_{i=1}^{n-1}(2i+3)^2$

15. $\sum_{i=1}^{n+1}(2i-1)^2$

16. $\sum_{i=1}^{n}(3i^2-2i+5)$

17. $\sum_{i=2}^{n-1}[(2n-i)^2+1]$

18. $\sum_{i=2}^{n-1}[(n-2i)^2+3]$

19. $\sum_{i=2}^{n-2}[(2n-3i)^2-n]$

3 Aplicação em Análise de Algoritmos

1. Ordenação por Seleção (*Selection Sort*)

Considere o código em C:

```
void selection_sort(int v[], int n) {
    int i, j, min, aux;
    for (i = 0; i < n - 1; i++) {
        min = i;
        for (j = i + 1; j < n; j++) {
            if (v[j] < v[min]) {
                min = j;
            }
        }
        if (min != i) {
            aux = v[i];
            v[i] = v[min];
            v[min] = aux;
        }
    }
}
```

1.a) Formule o somatório que contabiliza o total de execuções da comparação entre elementos do vetor e determine sua **fórmula fechada**. O resultado difere entre o melhor e o pior caso?

1.b) Considerando como movimentação cada atribuição envolvendo elementos do vetor, expresse o somatório e a fórmula fechada do número de movimentações:

- No **melhor caso**.
- No **pior caso**.

2. Ordenação por Bolha (*Bubble Sort*)

Considere o código em C com flag de troca para otimização:

```
void bubble_sort(int v[], int n) {
    int i, j, aux, trocou;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
            if (v[j] > v[j + 1]) {
                aux = v[j];
                v[j] = v[j + 1];
                v[j + 1] = aux;
            }
        }
    }
}
```

2.a) Número de comparações de entre elementos do vetor:

- Formule e resolva o somatório para o **melhor caso**.
- Formule o somatório e encontre a fórmula fechada para o **pior caso**.

2.b) Número de movimentações de chaves (3 atribuições a cada troca):

- Determine o total no **melhor caso**.
- Formule a somatória e encontre a fórmula fechada no **pior caso**.

3. Ordenação por Inserção (*Insertion Sort*)

Considere a implementação padrão em C:

```
void insertion_sort(int v[], int n) {
    int i, j, chave;
    for (i = 1; i < n; i++) {
        chave = v[i];
        j = i - 1;
        while (j >= 0 && v[j] > chave) {
            v[j + 1] = v[j];
            j--;
        }
        v[j + 1] = chave;
    }
}
```

3.a) Número de comparações de elementos:

- Formule o somatório e encontre a fórmula fechada no **melhor caso**.
- Formule o somatório e determine a fórmula fechada no **pior caso**.

3.b) Número de movimentações de elementos:

- Conte as movimentações envolvendo elementos do vetor no **melhor caso**.
- Formule o somatório que inclui o deslocamento de elementos dentro do laço while e resolva a fórmula fechada no **pior caso**.

4 Variações de Algoritmos de Ordenação

Questão 1. Adapte o algoritmo de **Ordenação por Seleção** (*Selection Sort*) para ordenar diretamente os elementos de uma **fila circular** em um arranjo de capacidade N. A fila deve ficar em **ordem crescente do ponteiro *primeiro* até o ponteiro *último***.

Exemplo: Considere uma fila circular com capacidade $N = 6$, onde primeiro $= 4$, ultimo $= 2$ e número de elementos $n = 4$:

Índice	0	1	2	3	4	5
Elemento	**52**	**20**	—	—	**38**	**14**
Ponteiros			↑ *ultimo*		↑ *primeiro*	

A sequência lógica de elementos do *primeiro* ao *último* é dada pelos índices $[4, 5, 0, 1]$, correspondendo aos valores:

Fila Original: [38, 14, 52, 20]

Após a execução do Selection Sort, as posições físicas do vetor são reorganizadas de modo que a leitura a partir de *primeiro* resulte em ordem crescente:

Fila Ordenada: [**14**, **20**, **38**, **52**]

Pede-se:

a) Desenhe o estado do vetor e os índices ao final de cada iteração do algoritmo para o exemplo fornecido.

b) Implemente em C/C++ a função:

```
void selectionSortFilaCircular(int array[], int primeiro, int ultimo, int N);
```

c) Demonstre como mapear o k-ésimo elemento lógico da fila (com $0 \leq k < n$) para o índice físico real no vetor usando o operador `%`.

Questão 2. Uma **fila de prioridade** estática pode ser implementada utilizando um arranjo contíguo mantido sempre ordenado de forma crescente. Dessa forma, a remoção do elemento de maior prioridade (menor chave) pode ser feita eficientemente.

Considere a seguinte definição para a estrutura da fila:

```
#define MAXTAM 100

typedef struct {
    int vetor[MAXTAM];
    int TAM;   // Capacidade maxima da fila
    int quant; // Quantidade atual de elementos armazenados
} FILA;
```

Sabendo que os elementos atualmente armazenados em `F->vetor` já se encontram ordenados de forma crescente, a operação de inserção de um novo elemento x deve manter esse invariante de ordenação.

Pede-se:

a) **Implementação:** Escreva o código completo da função `void inserir(int x, FILA *F)` em C/C++. O seu código deve ter complexidade de $\Theta(n)$. A função deve:

b) **Análise de Complexidade:** Analise a função implementada e determine:

- A complexidade de tempo de **melhor caso** e em qual cenário ela ocorre.
- A complexidade de tempo de **pior caso** e em qual cenário ela ocorre.

Questão 3. Descreva um algoritmo de tempo $\Theta(n \lg n)$ que, dado um conjunto S de n inteiros e um outro inteiro x, determine se existem ou não dois elementos em S cuja soma seja exatamente x.

Questão 4. O **Selection Sort** tradicional busca o menor elemento da parte não ordenada do vetor e o coloca no início dessa região.
Implemente uma variação desse algoritmo que faça o processo contrário: a cada iteração, percorra a parte não ordenada do vetor, encontre o maior elemento e troque-o com o elemento da última posição dessa região. Após cada iteração, a região não ordenada deve diminuir da direita para a esquerda, até que todo o vetor esteja ordenado em ordem crescente.

Questão 5. Considere os dois arrays de inteiros abaixo, ambos já ordenados em ordem crescente:

$$A = [2, 5, 8, 12, 16]$$

$$B = [1, 3, 7, 10, 14, 18]$$

Implemente uma função *merge* que receba esses dois arrays e produza um terceiro array contendo todos os elementos dos arrays originais em ordem crescente. A função deve realizar essa operação com custo de tempo $O(n + m)$, sem utilizar funções prontas de ordenação.

Questão 6. Considere os dois arrays de inteiros abaixo, ambos já ordenados em ordem crescente:

$$A = [2, 5, 8, 12, 16]$$

$$B = [1, 3, 5, 10, 12, 18]$$

Faça, em C, uma função que receba os dois arrays e retorne um novo array contendo os elementos em comum entre eles. A função também deve informar, por meio de um ponteiro passado como parâmetro, a quantidade de elementos presentes no array retornado. A solução deve possuir custo de tempo $O(n + m)$, onde n e m são os tamanhos dos arrays, respectivamente.

5 Pilhas e Filas

Questão 1. Mostre como implementar uma fila usando duas pilhas. Analise o tempo de execução das operações em filas.

Questão 2. Mostre como implementar uma pilha usando duas filas. Analise o tempo de execução das operações em pilhas.

Questão 3. Explique como implementar duas pilhas em um único arranjo $A[1..n]$ de tal modo que nenhuma delas sofra um estouro a menos que o número total de elementos em ambas as pilhas juntas seja n. As operações PUSH e POP devem ser executadas no tempo O(1).

6 Lista Encadeada

Questão 1. Implemente uma pilha usando uma lista simplesmente ligada L. As operações PUSH e POP ainda devem demorar o tempo O(1).

Questão 2. Implemente uma fila por meio de uma lista simplesmente ligada L. As operações ENFILEIRAR e DESENFILEIRAR ainda devem demorar o tempo O(1).

Questão 3. Implemente as operações INSERIR, DELETAR e PESQUISAR usando listas circulares simplesmente ligadas. Quais são os tempos de execução dos seus procedimentos?

Questão 4. Dê um procedimento não recursivo de tempo O(n) que inverta uma lista simplesmente ligada de n elementos. O procedimento só pode usar armazenamento constante além do necessário para a própria lista.

Questão 5. A operação em conjuntos dinâmicos UNION utiliza dois conjuntos disjuntos S_1 e S_2 como entrada e retorna um conjunto $S = S_1 \cup S_2$ que consiste em todos os elementos de S_1 e S_2. Os conjuntos S_1 e S_2 são normalmente destruídos pela operação. Mostre como suportar UNION no tempo O(1) usando uma estrutura de dados de lista adequada.